

Kiosk Vision AI

System Architecture — Working Document

Prepared by: Person 5 (Backend, Payments & Accessibility Output)

Purpose of This Document

This is the locked architecture for Kiosk Vision AI, an AI-powered accessible kiosk assistant, decided one piece at a time rather than all at once. Each decision below includes what was chosen and why, so any team member — or a judge — can see the reasoning, not just the conclusion. Two areas (computer vision pipeline specifics, and detailed frontend component design) are intentionally left as draft recommendations, not locks, since they belong to the teammates who will actually build them (Person 3 for CV, Person 4 for frontend implementation detail).

1. Core System Shape

1.1 Deployment Format — Browser-based Kiosk

Decision

The kiosk runs as a full-screen, kiosk-mode-styled React web app in the browser, not on dedicated hardware (no Raspberry Pi / tablet build).

Why

- Team is mostly learning their stack during the hackathon — physical hardware adds camera-driver, mic-input, and no-live-reload pain on top of that.
 - A full-screen browser app looks and feels like a kiosk to judges while keeping the team's normal, fast web dev workflow.
-

1.2 Backend Structure — Modular Monolith

Decision

One FastAPI application, internally split into isolated modules (/voice, /vision, /gaze, /orders, /payment) each owned by one person, sharing a common core/ for session state and models.

Why

- Avoids microservices overhead (service discovery, inter-service networking, CORS-per-service) that a 5-person team doesn't have time to wire up.
- Still gives each teammate clean code ownership so people aren't stepping on each other's routes.
- Single process to run and demo — no juggling multiple servers on judging day.

Trade-off / honest caveat

A hard crash in one module's unhandled exception could take the whole process down — mitigated by mandatory try/except at every router boundary (see Section 5).

1.3 Real-Time Communication — WebSockets

Decision

A single WebSocket connection per kiosk session carries all live backend-to-frontend events (captions, mode changes, payment status, screen narration, gaze events, errors). Raw microphone audio is transported separately (chunked upload to the STT provider), never mixed into this event channel.

Why

- Multiple different event types need to reach the frontend continuously and unpredictably — exactly what WebSockets are built for.
- Keeps latency low for the features where it matters most (live captioning).
- Separating audio transport from the event channel prevents Person 1 (voice) and Person 4 (frontend) from colliding on the same connection for two very different jobs.

Trade-off / honest caveat

Polling was ruled out (too laggy for captions); SSE was ruled out (one-directional only — gaze and voice both need frontend-to-backend flow too).

2. Data & Access

2.1 Database — SQLite, Postgres-ready

Decision

SQLite for development and the hackathon build, accessed entirely through SQLAlchemy (no raw SQLite-specific queries), so switching to Postgres later is a one-line connection-string change.

Why

- Zero setup tax (no Docker, no server to run) for genuinely simple, relational data: sessions, orders, order items, escalation logs.
- Still gives a true, credible scalability story to judges: "we run SQLite in dev, this is a one-line swap to Postgres for production."

Trade-off / honest caveat

MongoDB was considered and ruled out — the data is clean and relational (order belongs to session, item belongs to order), not unstructured, so document storage would be solving a problem that doesn't exist.

2.2 What Gets Persisted

Decision

Sessions, orders, order items, and escalation/handoff logs are persisted. Live captions and transcripts are ephemeral — shown via WebSocket, never written to the database.

Why

- Caption scrollbar was never a scoped feature; persisting it would be scope creep with no demo payoff.
 - The real accessibility need (following along live) is satisfied by live display alone.
-

2.3 Authentication — Anonymous Kiosk, Authenticated Attendant

Decision

Kiosk users get a session ID only, no login. The human-attendant dashboard (which receives escalations) requires simple authentication (JWT or a hackathon-speed shared token).

Why

- Requiring login before a walk-up user can order would directly contradict the product's accessibility goal.
 - The attendant dashboard is the only real security surface worth protecting — it can see live user sessions and respond to handoffs.
-

3. Payments

3.1 Payment Architecture — Real Stripe Test Mode + Real Backend State Machine

Decision

Stripe's test-mode API handles the simulated charge (test card numbers, no real money moves). The backend owns a genuine order-payment state machine: pending → processing → paid / failed. Stripe's async webhook confirmation triggers a WebSocket push to the frontend for the audio/visual confirmation.

Why

- The accessibility feature here (read the amount aloud, guide step by step, confirm before charging) needs a real state machine to react to — there's no meaningful audio guidance without real state transitions.
- Team explicitly chose this to learn real Stripe integration, and Stripe test mode costs nothing — test cards never move real money.

Trade-off / honest caveat

Adds real setup time (account, API keys, webhook handling) versus a pure fake-timer gateway — accepted deliberately since the team has runway and wants the learning.

4. AI Orchestration

4.1 Orchestration Model — Lightweight Rules Aggregator (in-process)

Decision

One shared session-state object inside the backend (not a separate networked service) holds simple signals from every pipeline: voice_confidence, voice_low_confidence_streak, gaze_stability, idle_seconds, failed_tap_count. A small, explicit rules function (plain threshold checks, not weighted ML scoring) reads that state and fires mode_change or handoff_triggered events.

Why

- Features 3 (Adaptive UI), 7 (Automatic Need Detection), and 10 (Human Handoff) all require a combined, cross-modality view — no single pipeline can make these calls alone.
- Kept in-process (not a separate microservice) to avoid a network hop on every event — important because gaze data can fire several times a second.
- Fully explainable to judges: every trigger is a plain, statable rule, e.g. "idle > 8s and failed_tap_count >= 3 → simplified mode."

Trade-off / honest caveat

High-frequency signals (gaze) are throttled/debounced (evaluated on the order of every 500ms–1s) before reaching the rules engine, rather than on every raw tick — this is what keeps latency low despite being a real coordination layer.

4.2 How Each Pipeline's "Score" Is Actually Calculated

Decision

Voice confidence comes directly from the STT provider's own output (no custom model needed). Gaze stability is a simple statistical calculation — variance of the last ~10 gaze coordinates (tight cluster = stable, wide scatter = unstable). Idle time and failed-tap count are plain counters/timestamps, no calculation involved. Screen-understanding (VLM) doesn't feed an ongoing score — its only orchestration-relevant signal is call failure/timeout.

Why

- Every signal is explainable end-to-end — no black-box scoring, which matters if a judge asks "how exactly did that decision get made."

- Keeps compute cost near zero, which matters for the in-process, low-latency design above.

5. Reliability & Error Handling

5.1 Graceful Degradation Pattern

Decision

Every external/pipeline call (STT, LLM, TTS, VLM, gaze, Stripe) is wrapped in try/except at the router boundary. On failure, the backend pushes an error WebSocket event (with a source field) instead of crashing or hanging. Failures increment counters that feed the same orchestrator rules engine from Section 4 — repeated failures in voice or payment automatically trigger handoff_triggered.

Why

- No pipeline failure should be able to cascade and take down the others — a broken VLM call must never block voice ordering, which must never block payment.
- Reuses existing infrastructure (orchestrator + WebSocket) rather than building a separate error-handling system.
- Gives a strong, honest answer to "what happens if X breaks in production": pipelines fail independently, and repeated failures automatically escalate to a human.

6. API & Event Contracts

6.1 REST Convention

Decision

Resource-based REST, scoped under /sessions/{id}/..., since nearly everything in the product happens within one kiosk session. Examples: POST /sessions, GET /sessions/{id}, POST /sessions/{id}/orders/items, POST /sessions/{id}/payment/initiate, POST /sessions/{id}/handoff.

Why

- One consistent, guessable pattern across every module, so a teammate can predict an endpoint's shape without checking docs.

6.2 WebSocket Event Envelope

Decision

Every event, regardless of which pipeline sends it, uses the same outer shape: { type, session_id, payload, timestamp }. The frontend routes purely on type; payload structure is free to differ per type.

Why

- type lets the frontend dispatch to the right handler without inspecting contents.
- timestamp helps debug event-ordering issues live and lets the UI handle out-of-order arrival gracefully.
- Backend (Person 5) owns and maintains the master list of event types, since it triggers most of them — this is the contract everyone else builds against.

Event type catalog

Event type	Fired by	Purpose
caption	Backend / TTS	Live text of what the kiosk is saying

voice_transcript	Voice pipeline	What the user said + STT confidence
order_updated	Backend	Order items / total changed
mode_change	Orchestrator	Switch UI layout (voice_first / simplified_ui / gaze_active)
payment_status	Backend / Stripe webhook	processing / paid / failed
handoff_triggered	Orchestrator / manual	Escalate to human attendant
screen_narration	Vision-language pipeline	Description of current screen
gaze_event	Gaze pipeline	Dwell-selection trigger
processing	Any pipeline	Immediate ack while a request is in flight
error	Any pipeline (try/except)	Non-fatal failure, source + message

7. Frontend Contract (for Person 4)

7.1 Mode Switching = Layout Swap, Not a Style Toggle

Decision

mode_change events (voice_first, simplified_ui, gaze_active) should trigger a structurally different component tree, not just a CSS/font-size change.

Why

- Feature 3's "step-by-step, icon-first, one-choice-at-a-time" layout is structurally different from the normal ordering grid — a toggle can't produce that.
-

7.2 Reactive + One Sync Point

Decision

Frontend primarily reacts to WebSocket events, but also calls GET /sessions/{id} once on page load and on reconnect, to restore full current state instead of waiting for the next event to happen to arrive.

Why

- Protects the demo against a refreshed tab or a WebSocket reconnect leaving the UI blank or stale mid-demo.
-

7.3 Immediate Acknowledgment via processing Events

Decision

A processing event fires the instant the backend starts handling a request (e.g. voice input received), before the real result is ready.

Why

- A blind/low-vision user has no visual loading spinner — going silent while a pipeline runs reads as "nothing happened." An immediate acknowledgment ("got it, one moment") preserves trust in the interface.
-

8. Deployment

8.1 Hosting — Frontend on Vercel/Netlify, Backend on Railway

Decision

Frontend deployed to Vercel or Netlify; backend (FastAPI + WebSocket) deployed to Railway.

Why

- Gives a real, judge-accessible URL and "this actually deploys" credibility beyond a laptop-only demo.

Trade-off / honest caveat

Free-tier cold starts on Railway are a real risk (30–60s wake-up on a cold hit). Mitigated by: (1) manually warming the backend a few minutes before the judging slot, (2) keeping a localhost fallback ready to pivot to instantly if needed. CORS must be configured for the cross-domain frontend/backend split, and WebSocket support on Railway's free tier should be verified early, not discovered in week 6+.

9. Scalability & Future Extensibility (Judge Q&A Prep)

Not new build work — this is the honest, ready answer for "does this scale beyond a demo?"

- Database: SQLite now, but built on SQLAlchemy — moving to Postgres for production is a config change, not a rewrite.
 - Backend: modular monolith today; each module is already cleanly separated, so splitting any one into its own microservice later doesn't require a redesign.
 - Orchestration: rule-based today because it's fast and explainable; the same structure could have its thresholds replaced by a trained model later without touching the pipelines that feed it.
 - Multi-kiosk deployment: session-based design already isolates each kiosk's state, so horizontal scaling (more backend instances behind a load balancer, shared Postgres) is a natural next step, not a fight against the architecture.
 - Cost at scale: real AI API calls (STT, LLM, TTS) cost money per interaction at volume — a genuine production version would need cost-optimization work (caching common intents, possibly smaller self-hosted models for high-volume kiosks).
-

10. Deferred to Teammates

These are intentionally not locked here — they belong to the person who will actually build them, and should be discussed directly with them rather than decided top-down:

- Computer vision pipeline specifics (VLM choice for screen understanding, gaze estimation library/approach) — Person 3's call. The event contract they need to build against (screen_narration, gaze_event) is already defined in Section 6.
 - Detailed frontend implementation (state management library, component structure) — Person 4's call. The contract they need to build against (event envelope, mode-switch behavior, sync endpoint, processing events) is already defined in Sections 6–7.
-

11. Voice AI Pipeline (Person 1)

11.1 Speech-to-Text — Gladia

Decision

Gladia, used with streaming audio input.

Why

- Free tier (480 min/month) is generous enough for hackathon development and demo use.

- Native multilingual support including Hindi-English code-switching — directly matches the locked Phase 3 multilingual feature.
-

11.2 Intent Parsing — Small/Fast LLM Tier

Decision

A fast, cheap model class (GPT-4o-mini or equivalent), forced to return strict JSON output (action, item, quantity, modifiers) rather than free text.

Why

- Order-intent parsing is a narrow, well-defined task that doesn't need a top-tier model.
 - Strict JSON output keeps the pipeline predictable and avoids a second parsing step.
 - Lower latency matters directly — the user is waiting to hear a spoken confirmation.
-

11.3 Text-to-Speech — Amazon Polly (dev) → ElevenLabs (demo)

Decision

Development and iteration use Amazon Polly (generous free tier, good enough quality for testing). The final demo build swaps to ElevenLabs for premium voice quality. TTS is built behind a thin internal interface (`speak(text) → audio`) so the swap is a one-file change.

Why

- ElevenLabs' free tier (~10 min/month) is too tight for repeated development iteration but well-suited to a polished final demo.
 - Voice quality is part of the product for an accessibility-first voice interface, not a cosmetic detail — worth the swap for the moment judges are watching.
-

11.4 Audio Transport — Streaming

Decision

Audio streams continuously to the STT provider while the user speaks, rather than waiting for a complete recording before sending.

Why

- Team explicitly chose this for a more real-time feel and to learn streaming integration, with awareness of the added complexity (persistent connection handling, reconnect logic) versus simpler chunked upload.

Trade-off / honest caveat

This is the team's highest-complexity build item on the most load-bearing feature (every other AI feature depends on voice working) — worth extra buffer time and testing in weeks 1–4.
